

- Data synthesis, with targeted class ratio.
- Classes generated by thresholding the discriminant < 0 . (complex)

```

if os.path.exists('data.csv'):
    data = pd.read_csv('data.csv')
else:
    # Generate dataset
    n = 40000
    a = np.random.uniform(-10, 10, n)
    b = np.random.uniform(-5, 15, n)
    c = np.random.uniform(-20, 5, n)
    discriminant = b**2 - 4*a*c
    # if discriminant is negative, the equation has no real roots
    d = (discriminant < 0).astype(int)

    # Ensure class balance (20%-40% ones)
    ones_ratio = np.mean(d)
    target_min, target_max = 0.2, 0.4
    if ones_ratio < target_min or ones_ratio > target_max:
        zero_indices = np.where(d == 0)[0]
        one_indices = np.where(d == 1)[0]
        target_ones = int(n * np.random.uniform(target_min, target_max))

        if len(one_indices) > target_ones:
            drop_ones = np.random.choice(one_indices, len(one_indices) - target_ones, replace=False)
            d[drop_ones] = 0
        elif len(one_indices) < target_ones:
            add_ones = np.random.choice(zero_indices, target_ones - len(one_indices), replace=False)
            d[add_ones] = 1

    # Convert to DataFrame
    data = pd.DataFrame({'a': a, 'b': b, 'c': c, 'd': d})
    data.to_csv('data.csv', index=False)

data.head()

```

	a	b	c	d
0	5.853709	-1.019291	-7.280964	0
1	-3.686291	-2.617174	-15.475696	1
2	0.132778	4.913126	-11.657493	0
3	9.368222	11.338011	-4.024927	0
4	-2.426093	1.938251	-12.959698	1

```

# Split into train and test sets
feature_names = ['a', 'b', 'c']
X = data[feature_names]
y = data['d']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Use Stratified K-Fold Cross-Validation
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Standardize features for models that require it
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

- Data split into train/test.
- KFold CV used to estimate test metrics without data leakage.
- Data standardized for optimal fit.
- GridSearchCV used for best model params.

```

: log_reg_params = {
    "penalty": ["l1", "l2", "elasticnet", None], # Different regularization techniques
    "C": [0.0001, 0.001, 0.01, 0.1, 1, 10, 100], # Wide range of regularization strengths
    "solver": ["liblinear", "saga", "lbfgs", "newton-cg"], # Solvers for different penalties
    "max_iter": [100, 500, 1000], # More iterations for convergence testing
    "l1_ratio": [0.1, 0.3, 0.5, 0.7, 0.9] # Only applies when penalty='elasticnet'
}
warnings.filterwarnings('ignore')
log_reg_grid = GridSearchCV(LogisticRegression(random_state=42), log_reg_params, cv=kf, scoring='accuracy')
log_reg_grid.fit(X_train_scaled, y_train)
log_reg_grid.best_params_

```

```

: {'C': 1, 'l1_ratio': 0.1, 'max_iter': 100, 'penalty': 'l2', 'solver': 'lbfgs'}

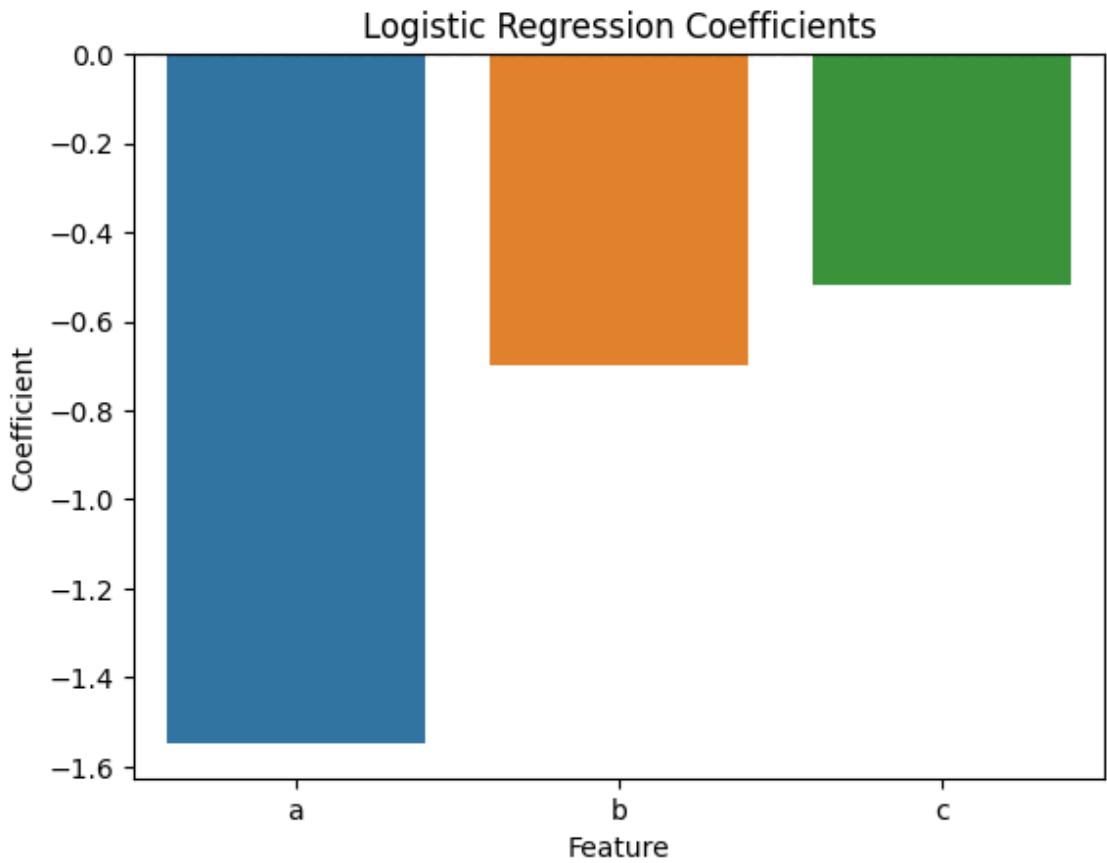
```

```

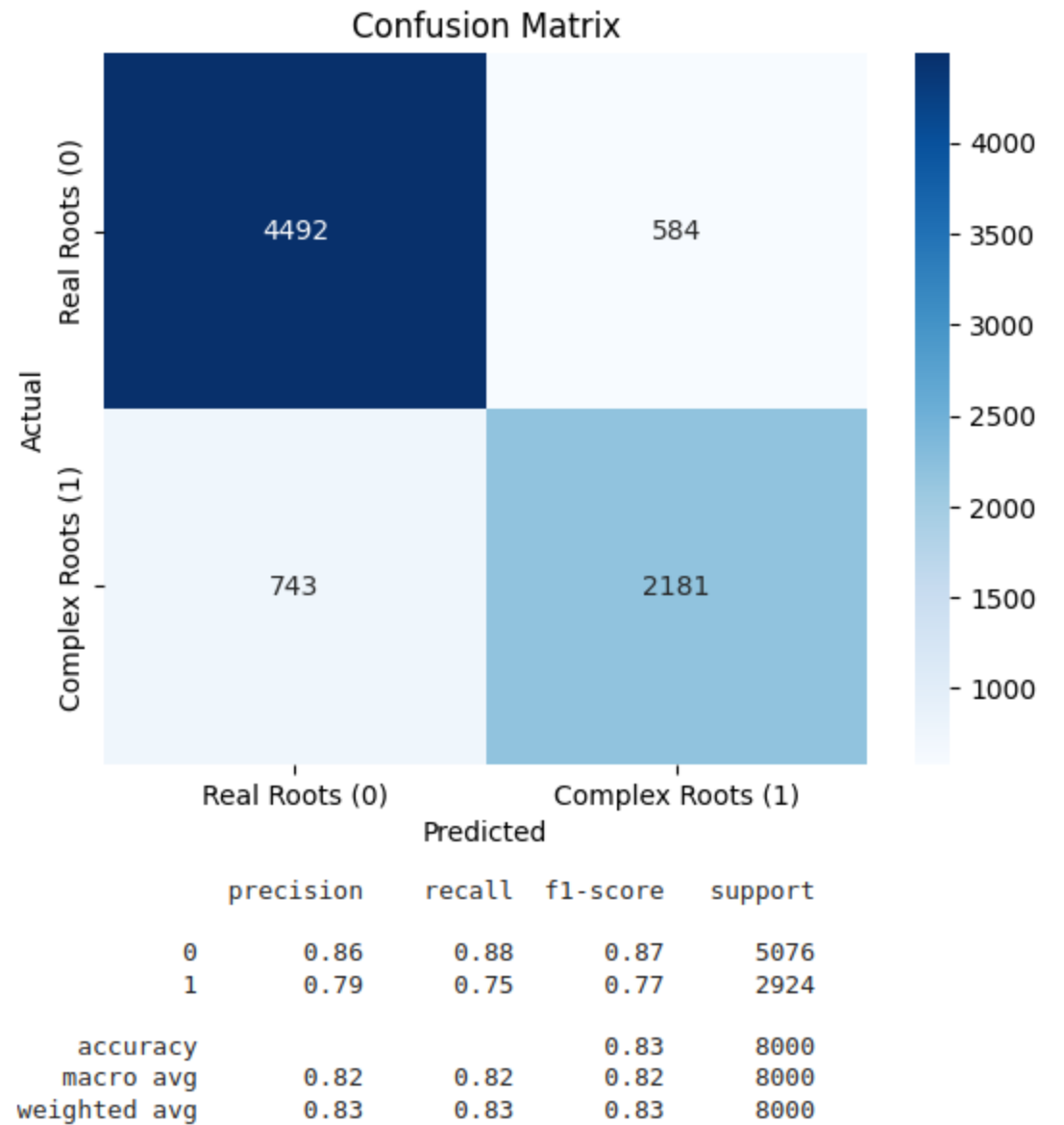
: lr = LogisticRegression(**log_reg_grid.best_params_, random_state=42)
  lr.fit(X_train_scaled, y_train)

```

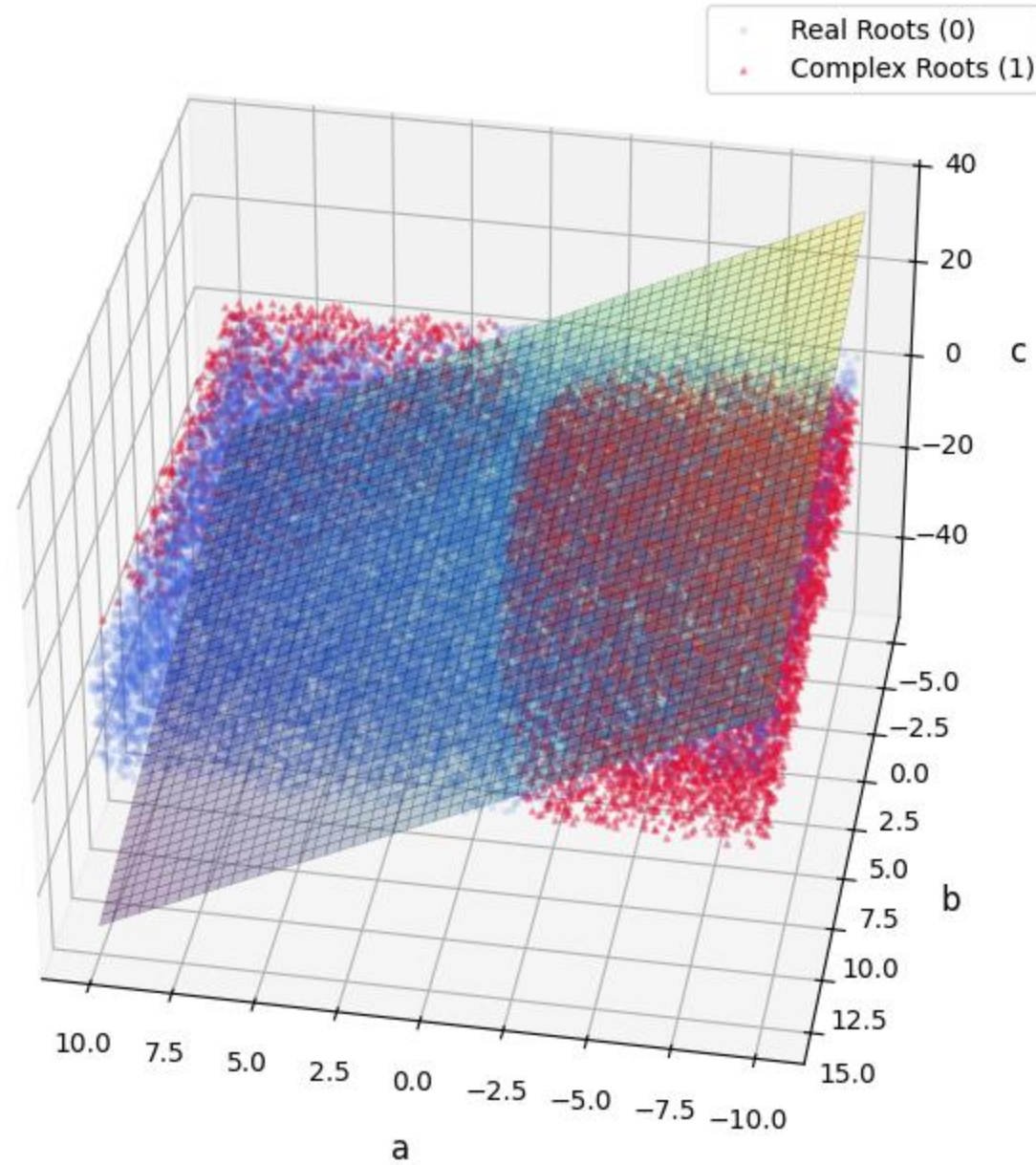
Feature	Coefficient	Odds Ratio
0a	-1.548260	0.212618
1b	-0.700065	0.496553
2c	-0.520587	0.594172



AUC: 0.82
Sensitivity: 0.75
Specificity: 0.88



Decision Boundary Visualization



```
param_grid = {
    "max_depth": [4, 5, 6], # Controls tree depth to prevent overfitting
    "min_samples_split": [1500, 2000, 2500], # Min samples needed to split a node
    "min_samples_leaf": [500, 600, 700] # Min samples per leaf node
}
```

```
dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=kf, scoring='accuracy')
dt_grid.fit(X_train, y_train)
dt_grid.best_params_
```

```
{'max_depth': 5, 'min_samples_leaf': 500, 'min_samples_split': 1500}
```

```
np.mean(accuracies)
```

```
np.float64(0.9391562499999999)
```

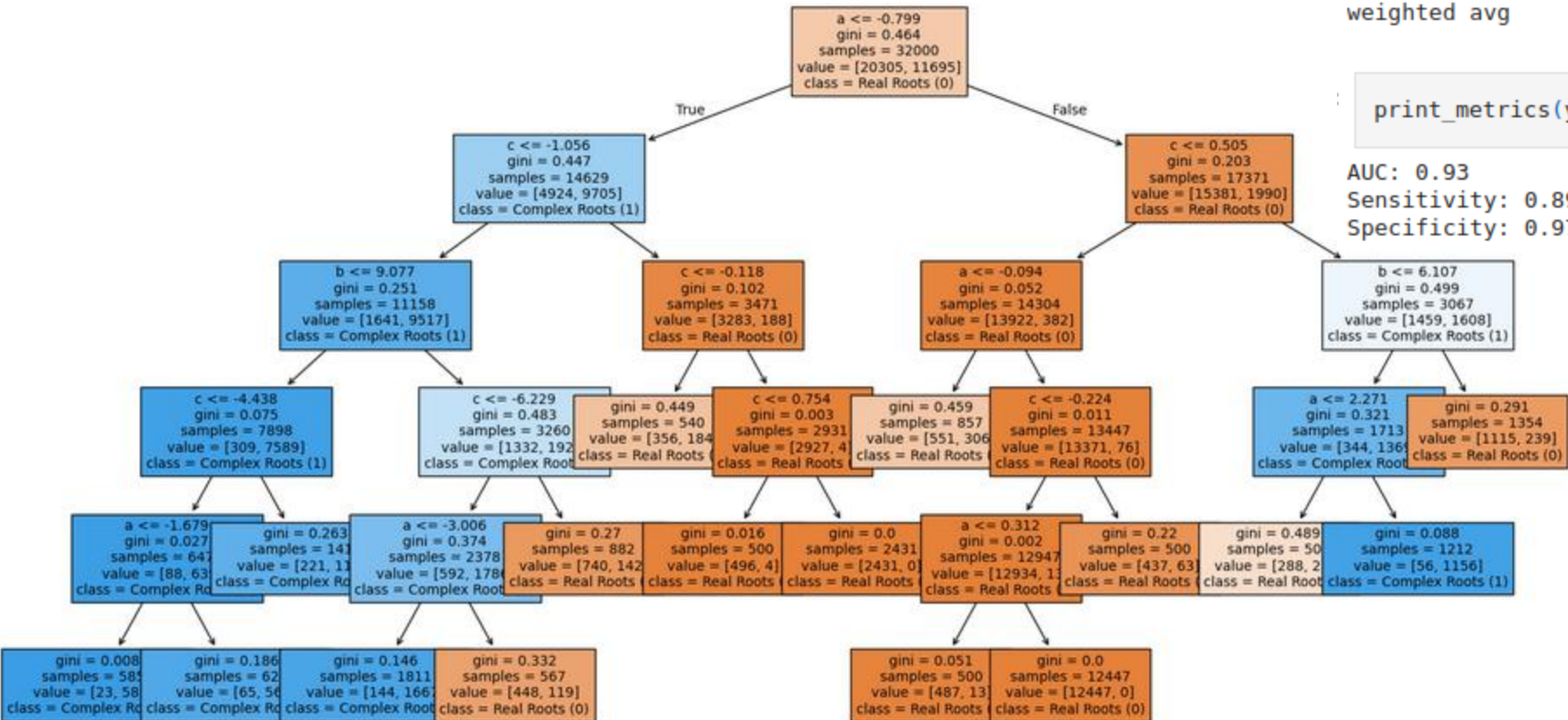
```
dt.score(X_test, y_test)
```

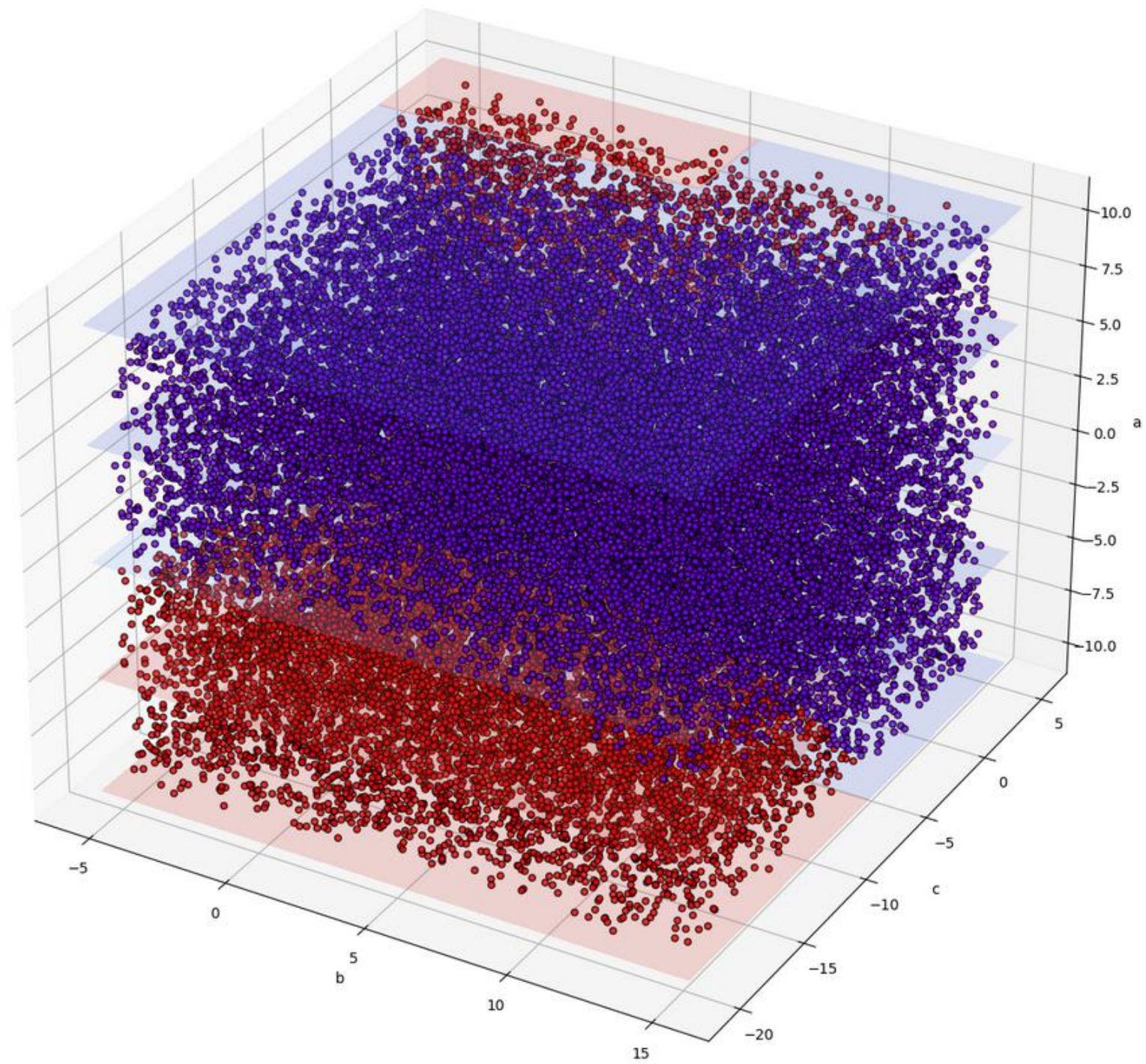
```
0.94025
```

	precision	recall	f1-score	support
0	0.94	0.97	0.95	5076
1	0.94	0.89	0.92	2924
accuracy			0.94	8000
macro avg	0.94	0.93	0.93	8000
weighted avg	0.94	0.94	0.94	8000

```
print_metrics(y_test, y_pred)
```

AUC: 0.93
Sensitivity: 0.89
Specificity: 0.97






```
y_pred = mlp.predict(X_test_scaled)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	5076
1	1.00	0.99	0.99	2924
accuracy			1.00	8000
macro avg	1.00	0.99	1.00	8000
weighted avg	1.00	1.00	1.00	8000

```
print_metrics(y_test, y_pred)
```

AUC: 0.99
Sensitivity: 0.99
Specificity: 1.00

